

Begonnen am	Donnerstag, 23. Januar 2025, 14:45
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 11:37
Verbrauchte Zeit	20 Stunden 51 Minuten

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Richtig

Nicht bewertet

Bitmanipulationen

Im Register R0 steht ein Wert. Schreiben Sie einen Befehl in ARM Cortex-M0 Assembler, um alle Bits in R0 zu invertieren.

MVNS R0,R0

Vergleichen Sie Ihre Antwort mit der Lösung. Falls sich diese nur durch zulässige Leerzeichen unterscheiden, ist Ihre Antwort korrekt.

Lösung

MVNS R0,R0

Frage 2

Richtig

Nicht bewertet

Bitmanipulationen

Schreiben Sie ein Code-Fragment in ARM Cortex-M0 Assembler, welches die Bits 10, 7 und 6 eines beliebigen Wertes im R0 invertieren. Nutzen Sie für die Bitmaske R1 und geben Sie diese in hexadezimaler Form, ohne führende Nullen an (Beispiel: 0x12).

Hinweise:

- *Bit 0 auf der rechten Seite ist das Bit mit der tiefsten Wertigkeit.*
- *Die restlichen Bits bleiben jeweils unverändert.*

Bits 10, 7 und 6 invertieren

LDR R1,=0x4C0

EORS R0,R1

Vergleichen Sie Ihre Antwort mit der Lösung. Falls sich diese nur durch zulässige Leerzeichen unterscheiden, ist Ihre Antwort korrekt.

Lösung

LDR R1,=0x4C0

EORS R0,R1 ;(oder EOR R0,R0,R1)

Frage 3

Teilweise richtig

Nicht bewertet

Bitmanipulationen

Schreiben Sie ein Code-Fragment in ARM Cortex-M0 Assembler, welches die Bits 3, 2 und 1 eines beliebigen Wertes im R0 auf logisch '0' setzt. Nutzen Sie für die Bitmaske R1 und geben Sie diese in hexadezimaler Form, ohne führende Nullen an (Beispiel: 0x12).

Hinweise:

- Benutzen Sie eine Variante mit **ANDS**
- Bit 0 auf der rechten Seite ist das Bit mit der tiefsten Wertigkeit.
- Die restlichen Bits bleiben jeweils unverändert.

a) Bits 3, 2 und 1 löschen (Variante unter Verwendung von **ANDS**)

LDR R1,=0xF1

ANDS R0,R1

b) Bits 3, 2 und 1 löschen (Variante **ohne** Verwendung von **ANDS**),

LDR R1,=0xE

BICS R0,R1

Vergleichen Sie Ihre Antwort mit der Lösung. Falls sich diese nur durch zulässige Leerzeichen unterscheiden, ist Ihre Antwort korrekt.

Lösung zu a)

LDR R1,=0xFFFFFFFF

ANDS R0,R1 ;(oder ANDS R0,R0,R1)

Lösung zu b)

MOVS R1,#0xE ;(oder LDR R1,=0xE)

BICS R0,R1

Frage 4

Richtig

Nicht bewertet

Bitmanipulationen

Schreiben Sie ein Code-Fragment in ARM Cortex-M0 Assembler, welches die Bits 8, 3 und 0 eines beliebigen Wertes im R0 auf logisch '1' setzt. Nutzen Sie für die Bitmaske R1 und geben Sie diese in hexadezimaler Form, ohne führende Nullen an (Beispiel: 0x12).

Hinweise:

- Bit 0 auf der rechten Seite ist das Bit mit der tiefsten Wertigkeit.
- Die restlichen Bits bleiben jeweils unverändert.

Vergleichen Sie Ihre Antwort mit der Lösung. Falls sich diese nur durch zulässige Leerzeichen unterscheiden, ist Ihre Antwort korrekt.

Lösung

LDR R1,=0x109

ORRS R0,R1 ;(oder OR R0,R0,R1)

Frage 5

Richtig

Nicht bewertet

Multiplikation mit einer Konstanten

In der folgenden Aufgabe wird eine Multiplikation $A * B$ durchgeführt. Finden Sie den Wert von A, indem Sie die Aufgabe schrittweise lösen. Nehmen Sie an $B = 3$. Beantworten Sie zunächst die folgenden Fragen:

1. In welchem Register wird das Resultat gespeichert? .
2. In welchem Register ist der Operand B gespeichert?

Tragen Sie in den Lückentext den Dezimalwert des
gefragten Registers nach der Ausführung ein:

MOVS R7,R0 Wert von R7?

LSLS R0,R0,#1 Wert von R0?

ADDS R7,R7,R0 Wert von R7?

LSLS R0,R0,#2 Wert von R0?

ADDS R7,R7,R0 Wert von R7?

LSLS R0,R0,#2 Wert von R0?

ADDS R7,R7,R0 Wert von R7?

Schliessen Sie aus der Anzahl der Shifts und den
Shift-Werten auf den Wert des Operanden A (8-
stellige Binärzahl mit führenden Nullen)

Bei den Addierschritten fällt auf, dass die Resultate immer
in Register R7 akkumuliert werden. Also muss R7 das
Ergebnis beinhalten.

Bei den Shifts fällt auf, dass immer R0 verschoben wird.
Also muss in R0 ursprünglich der Wert für den Operanden
B gespeichert worden sein.

Das ursprüngliche, unveränderte Byte wird zuerst in das
Akkumulationsregister R0 verschoben. Das LSBit des
Operanden A muss deshalb eine 1 sein.

Der erste Shift hat den Wert #1, danach wird addiert. Also
muss das 2. Bit des Operanden A eine 1 sein.

Der zweite Shift hat den Wert #2, also wurde das 3. Bit
ausgelassen und der Operand A war an dieser Stelle eine
0, gefolgt von einer 1

Der dritte Shift hat wieder den Wert #2, also wird das 5te
Bit ausgelassen und gefolgt von einer 1

Frage 6

Richtig

Nicht bewertet

**Multiplikation mit einer Konstanten
(Synthese)**

Vervollständigen Sie den Code eines Multiplizierers.

Es soll mit 23 multipliziert werden (erster Multiplikand). Wandeln Sie den Wert für 23 zunächst in eine 8-stellige unsigned Binärzahl um:

00010111

Tragen Sie in den Lückentext den Schiebewert x für den ersten LSLS ein, und ob die ADDS Zeile übersprungen oder genommen werden muss.

R7 soll am Ende das Produkt enthalten, zu Beginn ist $R7 = 0$.

R0 enthält zu Beginn den Wert des zweiten Multiplikanden (erster Multiplikand ist 23)

LSLS R0,R0,#x

Wert von x?

0

ADDS R7,R7,R0

Zeile ausführen

LSLS R0,R0,#1

Zeile ausführen

ADDS R7,R7,R0

Zeile ausführen

LSLS R0,R0,#1

Zeile ausführen

ADDS R7,R7,R0

Zeile ausführen

LSLS R0,R0,#1

Zeile überspringen

LSLS R0,R0,#1

Zeile ausführen

LSLS R0,R0,#1

Zeile überspringen

LSLS R0,R0,#1

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

ADDS R7,R7,R0

Zeile überspringen

Ergebnis bei, deshalb wird die Zeile übersprungen. usw.
Das 5. Bit ist eine 1 und wird wieder akumuliert.
Das 6., 7. und 8. Bit muss mit einer 0 aufgefüllt werden.